Here the build_stubbed strategy is used to create your project. The first spec group uses the factory as is, whereas the second group uses a key/value pair to set the associated tasks. (Very subtle note: you have to set the tasks attribute on Project rather than the project ID on Task because you're accessing it via the project, and build_stubbed only sets the association in one direction. You'll see more on this in a moment.)

All four of these build strategies allow you to pass key/value pairs to override the factory value for a given attribute. You can use this, for example, to create tasks with different sizes:

data/01/spec/models/project_spec.rb

```
describe "estimates" do
  let(:project) { FactoryBot.build_stubbed(:project,
    tasks: [newly_done, old_done, small_not_done, large_not_done]) }
  let(:newly_done) { FactoryBot.build_stubbed(:task,
    size: 3, completed_at: 1.day.ago) }
  let(:old_done) { FactoryBot.build_stubbed(:task,
    size: 2, completed_at: 6.months.ago) }
  let(:small_not_done) { FactoryBot.build_stubbed(:task, size: 1) }
  let(:large_not_done) { FactoryBot.build_stubbed(:task, size: 4) }

  it "can calculate total size" do
    expect(project).to be_of_size(10)
    expect(project).not_to be_of_size(5)
  end

  it "can calculate remaining size" do
    expect(project).to be_of_size(5).for_incomplete_tasks_only
  end
```

In this example, you're overriding the size and completed_at attributes of your tasks and assigning them to the project. Remember the last version of this code; you've gotten rid of the before hook, because you're now specifying the tasks as part of the factory.

When building the object from the factory, I consider it useful to explicitly list any attribute whose value is essential to the test. I do this even if the attribute has the same value as the factory default because I like having the explicit value in the test, where it can easily be seen. So, in this case, I might specify completed_at: nil for the not-done tasks in the setup.

All the build strategies will also yield the new object to a block in case you want to do more custom processing:

```
project = FactoryBot.build_stubbed(:project) do |p|
  p.tasks << FactoryBot.build_stubbed(:task)
end
```